IC Test Fixture YAML Test Scripts

YAML (YAML Ain't Markup Language) is a human-readable data serialization language. The syntax relies on proper indentation and primarily uses key-value pairs to create visually clean hierarchical data. By default, these files are encoded in UTF-8. Documentation for YAML can be found here: https://yaml.org/

Test scripts written in YAML are used to interface with the test fixture and test digital integrated circuits (ICs). These test scripts relay the operation of the IC and its expected performance under specified conditions. Customizable tests can be written to accommodate sequential and combinational logic.

Basic Syntax

YAML's primary data structure is based on key-value pairs. Keys must be scalar values, integers, strings, or floating-point decimals. A ':' is used to indicate the key-value pair relationship. All keys must be unique; otherwise, the value for a certain key will be overwritten with the most recent value for that key.

```yaml
A: 123

1: 4.5

3.14: pi
```

Figure 1.1. Examples of key-value pairs

Additionally, hexadecimal (0x) or binary (0b) can be used in place of integers; however, during parsing, they will be treated as unsigned integer literals. To keep them as a string, single or double quotation marks can be used to explicitly make them a string.

```yaml
0b1010: 21 # 10: 21

0xAB: 4 # 171: 4

"0b1010": 21 # 0b1010: 21

"0xAB": 4 # 0xAB: 4
```

Figure 1.2. Examples of non-base 10 integers

To create an array/list, either square brackets ([]), delimiting elements with a comma, or hyphens (-) with an extra indentation on subsequent lines can be used.

```
my list 1: [H, Z, L]

my list 2:
  - H
  - Z
  - L
```

Figure 1.3. Examples of creating a list

To create nested dictionaries, curly brackets ({}), delimiting key-value pairs with a comma, or chaining multiple key-value pairs with extra indentations based on the level of nesting needed on subsequent lines, can be used. When using key-value pairs, the highest key must have an empty value.

```
my dict 1:
  1: H
  2: A
  3:
    4: C

my dict 2: {1: H, 2: A, 3: {4: C}}
```

Figure 1.4. Examples of creating a nested dictionary

Any numbers relating to pins are restricted to a range of 1-20, the number of pins the hardware can support.

The supported voltages are within the set 1.8, 2.5, 3.3, 4.0, 4.5, and 5.0.

The supported input logic is within the set H, L, R, F, and X. R and F are respectively rising and falling edges. This will communicate the test fixture to pulse the specified edge. The test fixture only supports pulsing one of these signals at a time. X will default to logic LOW when used.

The supported output logic is within the set H, L, Z, and X. Z represents the high impedance output (Hi-Z), typically used for ICs with tri-state outputs.

Any IC with special outputs beyond the supported logic symbols can be tested by creating serial inputs and serial outputs to represent the expected output at a lower level.

Chip Info

The Chip Info section allows users to include any additional data regarding the chip under test. None of this information is verified or parsed by the software, as it does not affect how the chip is tested. The information listed here is to be added to the automatically generated PDF report after testing the device is completed. This is useful if the report is to be read by others who may not fully understand the contents of the report or would like additional testing metadata. This section is optional in the test script.

```
Chip Info:
  Name: 74HC00
  Manufacturer: Texas Instruments
  Logic: NAND
  Package: DIP-14
  Datasheet: https://www.ti.com/lit/ds/symlink/sn74hc00.pdf
```

Figure 2.1. Example of information about a chip.

Note: everything printed to the PDF report will be the default string representation in Python.

Global Parameters

The Global Parameters section lists all of the test parameters used across all tests in the test sections. These parameters include: VCC Pin, GND Pin, VCC Voltage, Input Low, Input High, Output Low, and Output High.

VCC Pin and GND Pin are, respectively, the power and ground pins used to power the IC during operation. Their values are literal integers. The VCC Voltage is the voltage level used to drive the chip; this voltage is also used as the default logic HIGH voltage during testing. This is expected to be a floating-point decimal or a list of floating-point decimals.

Input Low, Input High, Output Low, and Output High are the voltage thresholds the chip recognizes for an input or output of logic LOW and logic HIGH. Any output logic identified by the system is based on the given output voltage thresholds. Input Low and Input High are optional parameters that the user can provide. Their values are either a floating-point decimal or a list of floating-point decimals.

All lists should be parallel to each other, meaning they must be the same length, and elements at the same index correspond to each other during testing. Using lists allows users to test ICs at various supply voltage levels. This section is required to be in the test script.

```
Global Parameters:
  VCC Pin: 20
  GND Pin: 7
  VCC Voltage: 5
  Output Low: 0.33
  Output High: 3.84
```

Figure 3.1. Example of Global Parameters Section

```
Global Parameters:
  VCC Pin: 20
  GND Pin: 7
  VCC Voltage: [2.5, 3.3, 4.5, 5]
  Input Low: [0.75, 0.99, 1.35, 1.5]
  Input High: [1.75, 2.31, 3.15, 3.5]
  Output Low: [0.1, 0.1, 0.1, 0.1]
  Output High: [2.4, 3.2, 4.4, 4.9]
```

Figure 3.2. Example of Global Parameters Section using list values.

Note: Input Low and Input High are not actively used during parsing or testing.

Pin Map

The Pin Map section provides an abstraction layer between the test scripts and the physical pins of the IC. It allows users to map pin names to their corresponding socket positions (represented by numeric values). By using the Pin Map, test scripts can reference pins by name rather than by integer values, improving readability and maintainability when specifying which pins to drive or measure. This section is optional in the test script.

Note that VCC and GND pins do not need to be included in the Pin Map, as they are already defined in the Global Parameters section and are always driven during IC testing.

```
Pin Map:
  A1: 1
  A2: 4
  A3: 9
  A4: 12
  B1: 2
  B2: 5
  B3: 10
  B4: 13
  Y1: 3
  Y2: 6
  Y3: 8
  Y4: 11
```

Figure 4.1. Example of Pin Map section

Truth Table

The Truth Table section provides an abstraction layer for defining the logic used to drive and evaluate pins in the test scripts. It is represented as a list of dictionaries, where each key–value pair defines a logic grouping and its associated supported logic symbol. The names of these logic groupings are independent and do not need to match any names defined in the Pin Map or other sections of the test script. Each dictionary in the list must contain the same set of keys and the same number of key–value pairs. While the order of keys does not need to be identical across dictionaries, maintaining a consistent order is recommended for readability and easier visualization of the IC's truth table. Each dictionary represents a single row in the IC's truth table. This section is optional in the test scripts.

```
Truth Table:
  - {A: L, B: L, Y: H}
  - {A: L, B: H, Y: H}
  - {A: H, B: L, Y: H}
  - {A: H, B: H, Y: L}
```

Figure 5.1. Example of Truth Table section.

Note: Serial inputs or outputs are not supported as a value in the Truth Table.

Tests

The Tests section defines how the test fixture is controlled, specifying which pins to drive, which pins to measure, and the expected logic outputs. This section is structured as a dictionary of test cases. Each test case is identified by a unique name and contains two subkeys: Inputs and Outputs. Multiple test cases can be defined to cover different scenarios. Each test case is independent; inputs and outputs do not carry over between tests. This section is required in the test scripts.

.

```
Tests:
  test name 1:
    Inputs:
    Outputs:
  test name 2:
    Inputs:
    Outputs:
```

Figure 6.1. Basic structure of Tests section

I/O Formatting

The Tests section offers various support for different formatting of inputs and outputs, allowing pins and their outputs to be logically grouped together in a single line. The basic structure for formatting inputs the pin numbers or names, delimited by a comma, followed by the driving/expected logic. There are 3 types of formatting that can be used: single, map, and serial.

Single

With Single formatting, there is only one logic in the value of the pair. This logic is mapped to all the pins designated in the key. Optionally, a voltage may be supplied in the input command to use a different voltage from the VCC voltage when driving a logic HIGH signal. If a logic LOW is specified, the voltage is ignored. This type of command is useful when multiple pins with similar responsibilities require the same type of logic.

```
Tests:
  test name 1:
    Inputs:
      1: L
      4: H 3.3 # drives logic HIGH with 3.3V
    Outputs:
      2,3: H
      # equivalent to
      # 2: H
      # 3: H
```

Figure 6.2. Examples of a Single type command

Map

Using the Map formatting, an equal number of logic values and pins directly correspond to each other. The first pin listed will have the first logic value listed, the second pin will have the second logic value, etc. This type of command is useful for pins that have similar responsibilities but require different logic values. For example, select bits on a multiplexer, or n-bits used to represent integers. Additionally, this command also supports integers as the logic value. The integers assume big-endian notation, meaning the first pin will represent the most significant bit. Like Single, optionally, a voltage can be added to the input command to specify a different voltage than the VCC voltage for logic HIGH. If there is any logic LOW in the command, the voltage is ignored.

```
Tests:
  test name 1:
    Inputs:
      1,2: H,L 2.5 # 1 -> H @ 2.5V, 2 -> L
      3,4,5: 0b101 # 3 -> 1, 4 -> 0, 5 -> 1
    Outputs:
      6,7,8: L,L,L # 6 -> L, 7 -> L, 8 -> L
```

Figure 6.3. Examples of Map type command

Serial

The serial formatting allows pins to be driven with a series of logic values. This command is denoted by creating a list of logic values. All lists of logic values are parallel to each other, meaning that elements with the same index correspond to each other. During testing, inputs are driven with the value at index i, and the expected outputs are located at index i of the command.

Unlike separate test cases, the serial command will carry over inputs and states between each step.

```
Tests:
  test name 1:
    Inputs:
      1,2: [H,L]
      4: [X,R]
      3: [F,X]
    Outputs:
      7: [L,H]
      # Expects L when 1,2 -> H, 4 -> X, 3 -> F
      # Expects H when 1,2 -> L, 4 -> R, 3 -> X
```

Figure 6.4. Example of Serial type command

The lists do not need to be the same length; if one command is serial, the remaining inputs and outputs are padded to the longest serial command. They are padded using the last logic value specified in the command.

```
Tests:
  test name 1:
    Inputs:
      1,2: [H,L]  # padded to [H,L,L,L]
      4: [X,R]    # padded to [X,R,R,R]
      3: H        # padded to [H,H,H,H]
    Outputs:
      7: [L,H,Z,Z]
```

Figure 6.5. Serial type command with padding

The lists do not need to be the same length; if one command is serial, the remaining inputs and outputs are padded to the longest serial command. They are padded using the last logic value specified in the command. This command is useful for ICs with serial inputs and outputs, and ICs with outputs dependent on previous I/O. Unlike the previous command types, serial commands do not support a different voltage from the VCC voltage. Any logic value that uses a reference from a truth table is considered a serial command.

If Pin Map and/or Truth Table sections are included in the test scripts, the pins and logic values can be abstracted to the names of pins and logic values.

```
Tests:
  NAND 1:
    Inputs:
      A1: A
      B1: B
    Outputs:
      Y1 : Y
```

Figure 6.6 Abstracted commands using Pin Map and Truth Table references.